

# CASE STUDY: Urban Mobility Data

## Engineering Challenge

### Business Scenario

You are working as a Data Engineer for **UrbanMove**, a smart mobility analytics startup that analyses taxi, auto, and bike rides across a metropolitan city to improve transport planning.

UrbanMove wants to build a unified analytics platform to process millions of trip records daily.

You have been asked to:

- Generate synthetic trip data (1 million rows)
- Store them as CSV files
- Run Spark pipelines to clean, transform, enrich, aggregate & store data
- Use RDD, DataFrame, SQL, Parquet, MySQL, and text output formats

## 1 Dataset Specification (1 Million Records)

Each record represents one trip from point A to point B.

| Column Name   | Type      | Description            |
|---------------|-----------|------------------------|
| tripId        | Long      | Unique trip ID         |
| driverId      | Int       | Driver identifier      |
| vehicleType   | String    | "AUTO", "TAXI", "BIKE" |
| startTime     | Timestamp | Trip start time        |
| endTime       | Timestamp | Trip end time          |
| startLocation | String    | Area name              |
| endLocation   | String    | Area name              |

|                |        |                       |
|----------------|--------|-----------------------|
| distanceKm     | Double | Distance travelled    |
| fareAmount     | Double | Charged fare          |
| paymentMethod  | String | "CASH", "UPI", "CARD" |
| customerRating | Double | Rating 1.0 - 5.0      |

## 2 Scala Code to Generate 1 Million Records (CSV)

Have participants run this inside Dataproc/Spark notebook:

```
import java.io._
import scala.util.Random
import java.time._
import java.time.temporal.ChronoUnit
```

```
val file = new PrintWriter(new File("urbanmove_trips.csv"))
```

```
val areas = Array("MG Road", "Indira Nagar", "Koramangala", "Whitefield", "Marathahalli",
"HSR Layout", "BTM", "Jayanagar")
```

```
val vehicleTypes = Array("AUTO", "TAXI", "BIKE")
```

```
val paymentMethods = Array("CASH", "UPI", "CARD")
```

```
val rand = new Random()
```

```
def randomDateTime(): LocalDateTime = {
  val now = LocalDateTime.now()
  now.minusMinutes(rand.nextInt(100000))
}
```

```
file.write("tripId,driverId,vehicleType,startTime,endTime,startLocation,endLocation,distanceKm,fareAmount,paymentMethod,customerRating\n")
```

```
for(i <- 1 to 1000000){
  val start = randomDateTime()
  val duration = rand.nextInt(50) + 5
```

```
val end = start.plusMinutes(duration)
```

```
val distance = (rand.nextDouble() * 15 + 1).formatted("%.2f").toDouble
```

```
val fare = (distance * (rand.nextInt(10) + 10)).formatted("%.2f").toDouble
```

```
file.write(
```

```
  s"$i,${rand.nextInt(5000)},${vehicleTypes(rand.nextInt(3))},$start,$end," +
```

```
  s"${areas(rand.nextInt(areas.length))},${areas(rand.nextInt(areas.length))}," +
```

```
  s"$distance,$fare,${paymentMethods(rand.nextInt(3))}," +
```

```
  f"${rand.nextDouble() * 4 + 1}%.2f\n"
```

```
)
```

```
}
```

```
file.close()
```

This outputs:

**urbanmove\_trips.csv (1 million rows)**

## **3** Spark Tasks / Pipelines (10 Tasks)

These tasks mix **RDDs**, **DataFrames**, **SQL**, **Parquet**, **MySQL**, **text**.

### **Pipeline 1 — Load CSV using RDD**

**Goal:** Understanding raw RDD transformation.

**Tasks:**

- Load urbanmove\_trips.csv as RDD

- Filter trips with distance > 10 km

- Map to (vehicleType, distance)

- Save output as text file

## Pipeline 2 — Convert RDD → DataFrame Using Case Class

```
case class Trip(  
  tripId: Long,  
  driverId: Int,  
  vehicleType: String,  
  startTime: String,  
  endTime: String,  
  startLocation: String,  
  endLocation: String,  
  distanceKm: Double,  
  fareAmount: Double,  
  paymentMethod: String,  
  customerRating: Double  
)
```

Convert to DataFrame using `spark.createDataFrame`.

## Pipeline 3 — Clean DataFrame

Remove rows where `distanceKm <= 0`

Remove trips where `fareAmount < 0`

Validate that `startTime < endTime`

## Pipeline 4 — Derive a New Column (Trip Duration)

```
val df2 = df.withColumn("tripDurationMinutes",  
  (unix_timestamp($"endTime") - unix_timestamp($"startTime")) / 60 )
```

## Pipeline 5 — Aggregate Metrics

Average trip distance by vehicleType  
Revenue per day  
Top 5 most used routes

## Pipeline 6 — SQL Queries

Register as SQL table:

```
df2.createOrReplaceTempView("trips")
```

Write SQL:

```
SELECT vehicleType, COUNT(*) FROM trips GROUP BY vehicleType  
SELECT startLocation, endLocation, AVG(fareAmount) ...  
SELECT paymentMethod, COUNT(*) ...
```

## Pipeline 7 — Write DataFrame to Parquet

```
df2.write.mode("overwrite").parquet("parquet/trips_clean.parquet")
```

## Pipeline 8 — Read From Parquet and Re-Aggregate

Demonstrate optimization:

```
val pq = spark.read.parquet("parquet/trips_clean.parquet")  
pq.groupBy("vehicleType").avg("fareAmount").show()
```

## Pipeline 9 — Write to MySQL

Let them configure:

```
df2.write
  .format("jdbc")
  .option("url", "jdbc:mysql://HOST:3306/urbanmove")
  .option("dbtable", "trip_summary")
  .option("user", "root")
  .option("password", "pass")
  .mode("append")
  .save()
```

## Pipeline 10 — Export Analytics Report to Text

Take aggregated results and:

```
result.rdd.saveAsTextFile("reports/vehicle_revenue_report.txt")
```

## Learning Outcomes

Participants will learn:

- ✓ **Generating synthetic big data using Scala**
- ✓ **Loading CSV via RDD & DataFrames**
- ✓ **Transformations (map, filter, flatMap)**
- ✓ **Using case classes for schema**
- ✓ **Data cleaning & enrichment**
- ✓ **Aggregations using DataFrame + SQL**
- ✓ **Writing/reading Parquet**
- ✓ **Integrating Spark with MySQL**
- ✓ **Saving results as text files**
- ✓ **Design thinking for end-to-end pipelines**